

Composite Applications, Service Oriented Architecture and ESBs – Issues and Decisions

Ross Altman
Sun Microsystems

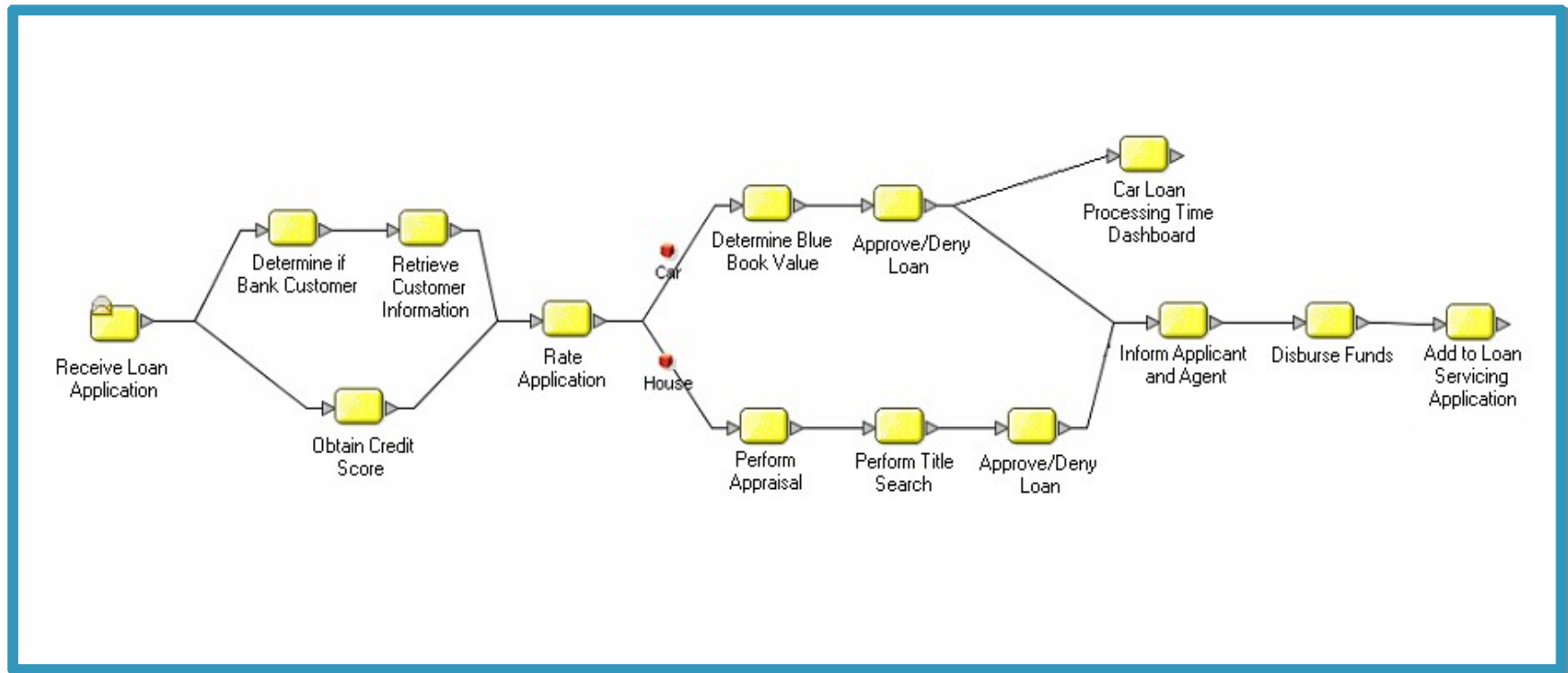
Agenda

- **Why should enterprises care about SOA?**
- How are open standards and open source playing a role in the enterprise transition to SOA?
- What are the biggest challenges around SOA?
- What is the future of SOA?

We deliver apps in a new way because we've changed the way we do business.

- We're already well into the era of implementing new ways of communicating with customers and their systems over the network.
- We must continue to support our current business models (and the technology that supports those business models) as we experiment with, and then deploy, new business models.
- Because we have to continue to support our existing business model, and because it would simply cost too much to pursue a “rip and replace” strategy, our infrastructure (including hardware and software) will not undergo a wholesale replacement.
- To the contrary, our investments must be incremental and additive.
- This leads us to the era of Composite Applications.

Composite Applications are process-oriented systems built “on top of” multiple existing systems.



Car Dealer Systems



Real Estate Agent Systems



Credit Bureau Systems



Internal Banking Systems



Car Valuation Application



Loan Rating Application



Funds Disbursement Application

How are Composite Applications different?

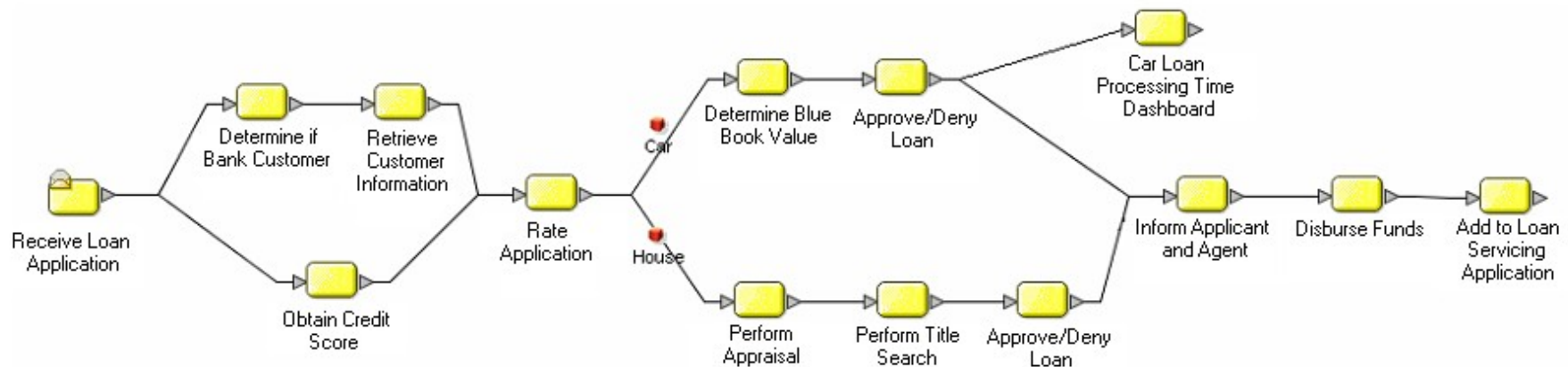
Legacy Apps

- 1 mainframe computer
- 1 user device
- 1 network connection
- 1 population of users
- 1 set of user requirements
- 1 user location
- 1 program
- ***1 program “owner”***

Composite Applications

- Many computers
- Many user devices
- Many network connections
- Many diverse populations of users
- Many different user requirements
- Many user locations
- Many programs
- ***Many program “owners”***

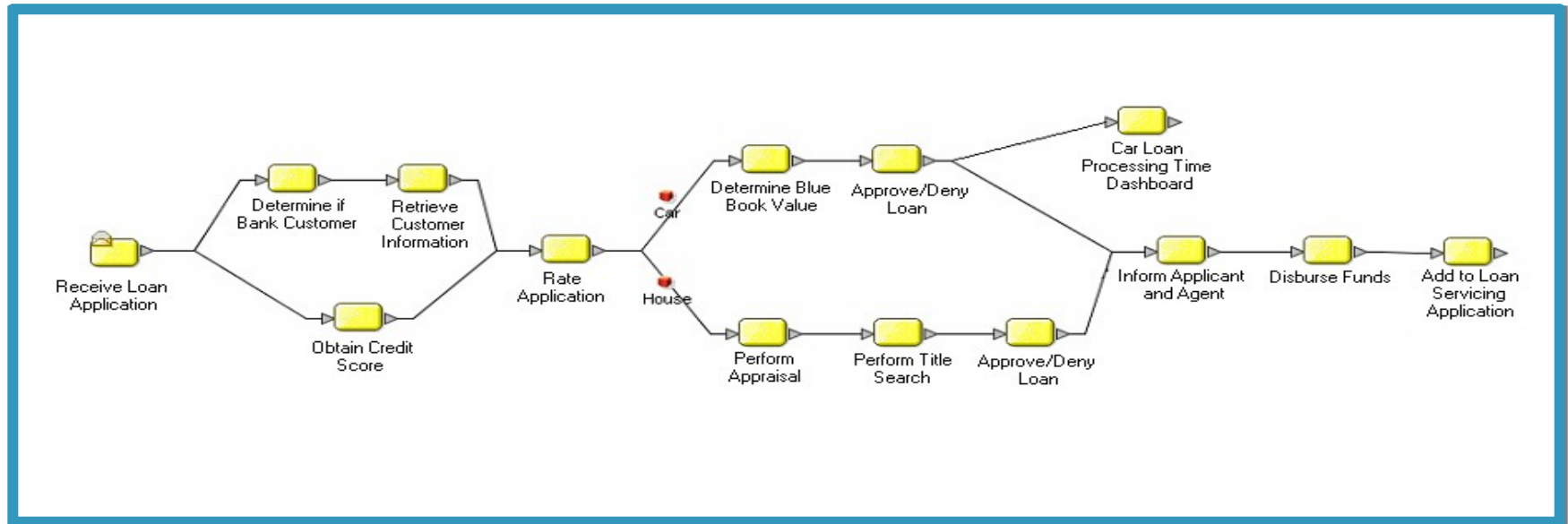
Composite Applications can include B2B, BPM, Workflow, Business Rules, Business Activity Monitoring, even ETL



Composite Applications Deliver Business Value

- Reduce personnel costs by automating process steps
- Reduce requirements for highly skilled decision-makers
- Decrease the frequency of “wrong decisions”
- Reduce process latency
- Track execution of multi-step processes to ensure process efficiency
- Track results of processes (through Business Activity Monitoring) to enable realtime process enhancement
- Automate some exception handling
- Thoroughly monitor and manage processes for compliance purposes

Composite Applications are built “on top of” multiple existing data and applications – each, potentially, implemented on a different architecture.



Car Dealer Systems



Real Estate Agent Systems



Credit Bureau Systems



Internal Banking Systems



Car Valuation Application

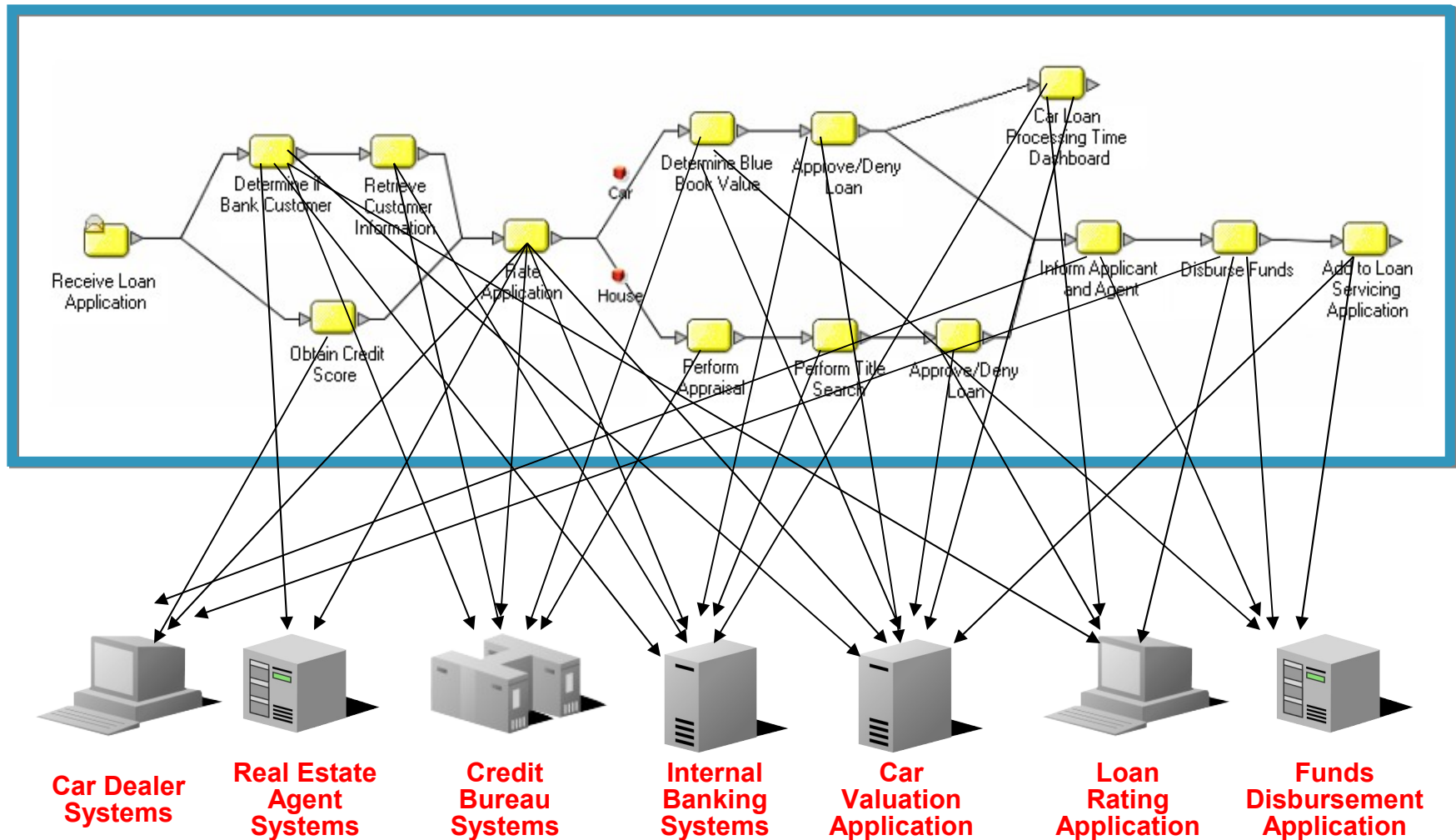


Loan Rating Application

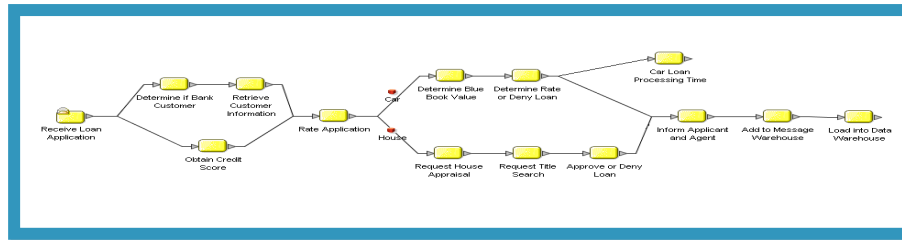


Funds Disbursement Application

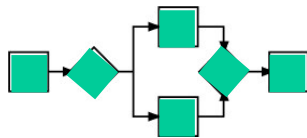
You could implement Composite Applications using point-to-point integration techniques. . .



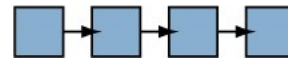
However, a Service Oriented Architecture provides us with multiple benefits for Composite Application development.



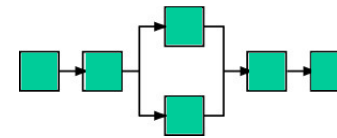
**Order Processing
Composite
Application**



Installation Scheduling



Process Customer Order



Bill Presentment/Payment

**Composed
Business
Service**



**Check Customer
Status**



**Verify Customer
Credit**



**Look-up
Customer Discount**



**Determine Product
Availability**



**Calculate
Shipping Charges**

**Elemental
Business
Services**



**Distribution
Systems**



**Materials
Management
Systems**



**Credit
Assessment
Applications**



**Manufacturing
Scheduling
Systems**



**Materials
Procurement
Application**



**Logistics
Management
System**



**Order
Processing
Application**

Agenda

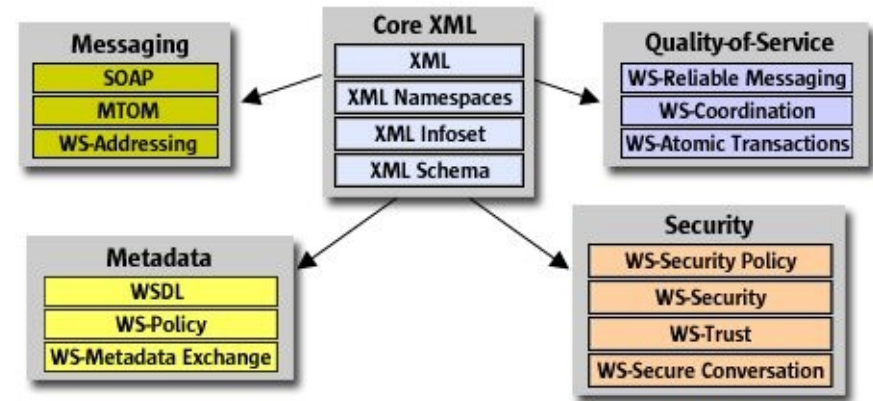
- Why should enterprises care about SOA?
- **How are open standards and open source playing a role in the enterprise transition to SOA?**
- What are the biggest challenges around SOA?
- What is the future of SOA?

Using Web Services to deliver true Plug-and-Play Interoperability

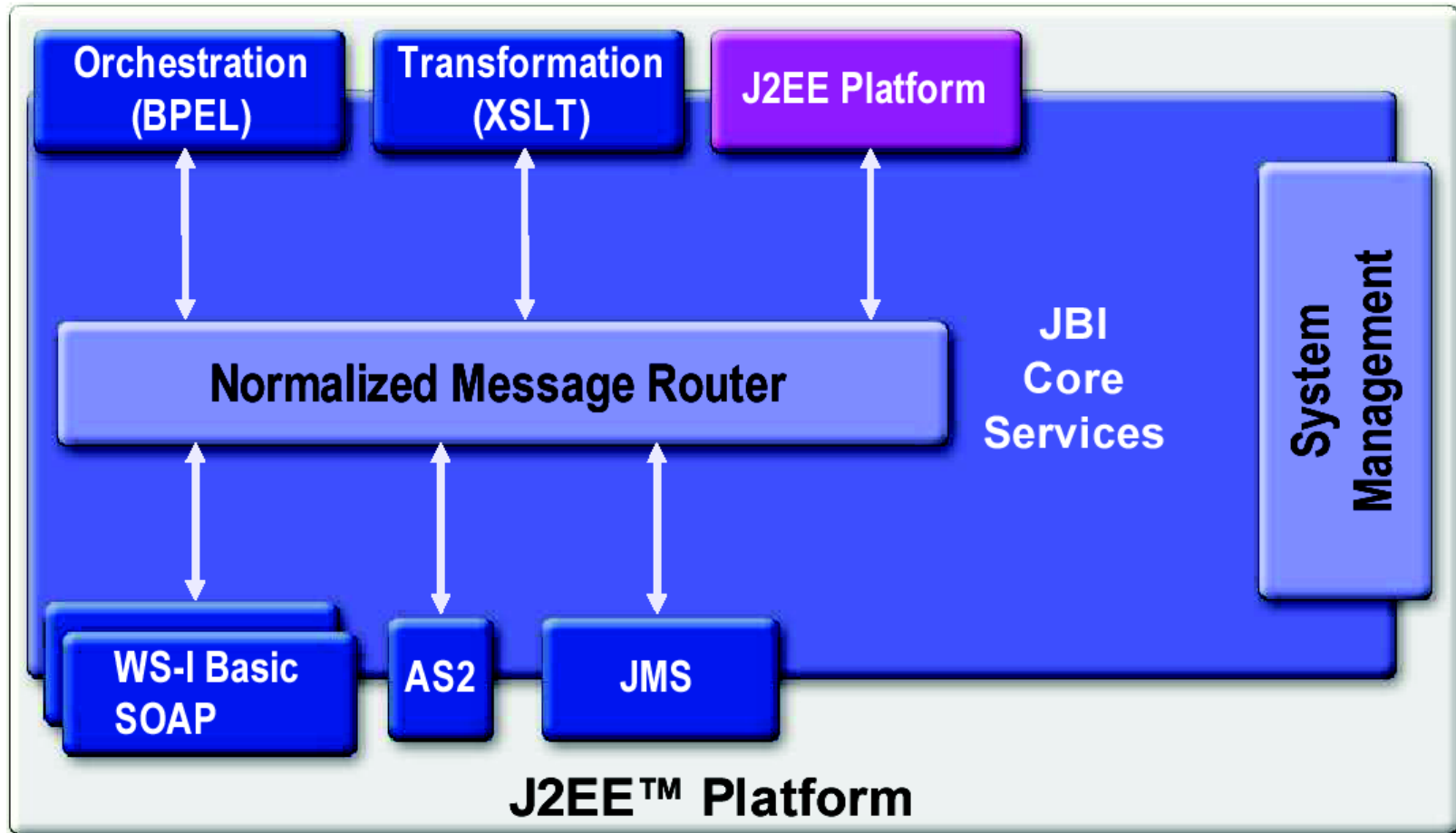
- **Projects Tango & Metro** – Multiple vendors collaborate on interoperable Web Services stacks
- Microsoft's implementation is in **Windows Communications Foundation** (formerly "Indigo")
- The Java implementation is in the open sourced **Project Metro**
- Economic coercion may ensure adoption by most major vendors – current Project Metro participants include BEA, IBM, JBoss and Sun

To offer interoperability:

- Everyone will need to provide "certified" interoperability with these stacks
- Plugfests will be the "certification mechanism" to ensure compatibility
- Snowballing adoption would generate the required coercion

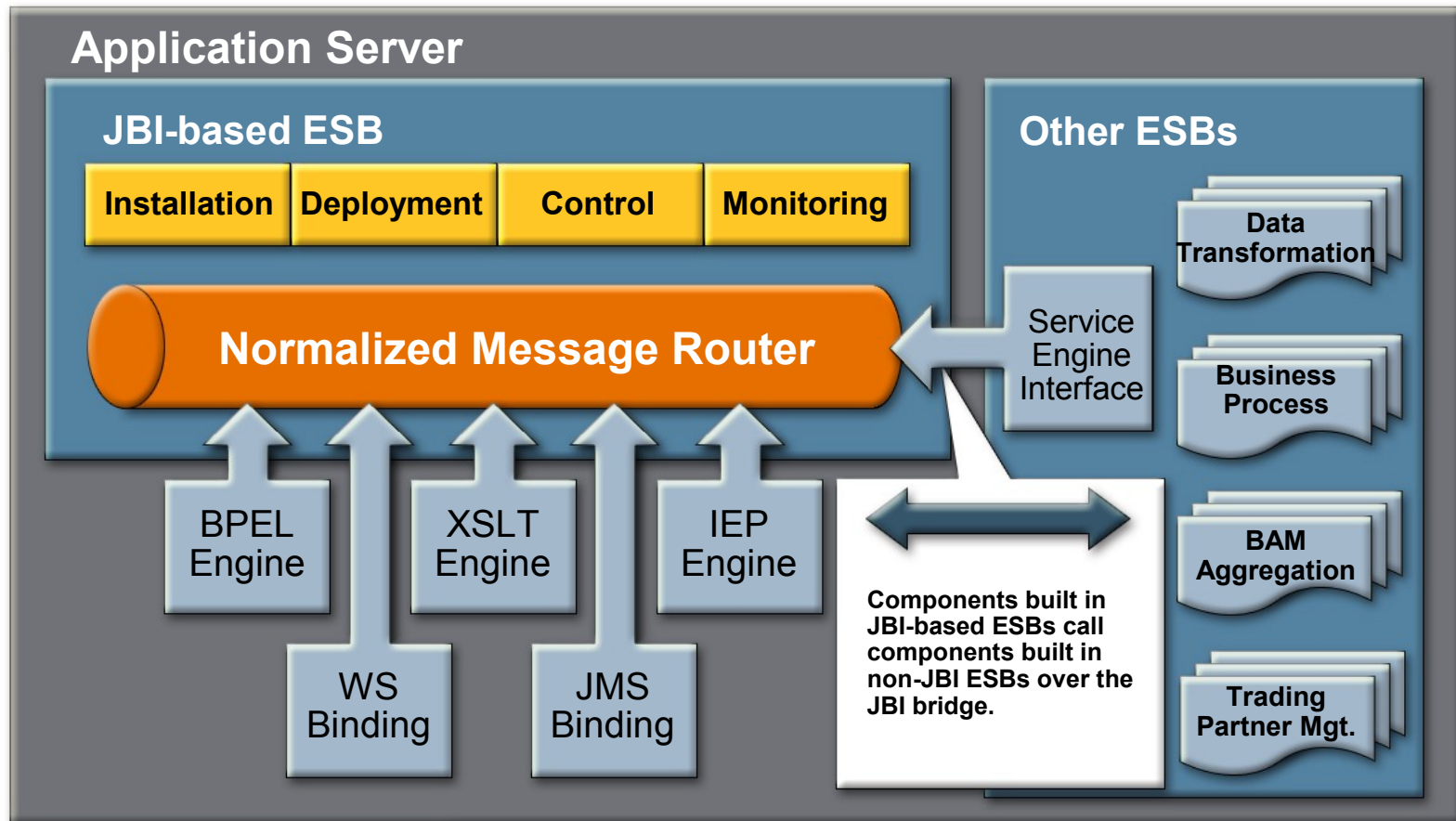


Portability Between JBI-based ESBs



Java Business Integration (JSR 208) provides a standard so components can plug into an ESB.

Integration of JBI-based and Non-JBI ESBs



Why Open Source?

- There's a Trojan Horse at the gate, should we bring it in?
 - Security is provided through extreme openness.
- Increased quality
 - Increased exposure of code causes code quality to go up
 - Public scrutiny causes bugs to be fixed faster
- Psst. Hey buddy, want some free software?
 - Free to download. Free to try. Free to use.
 - Pay for support when you need it.
- Barriers to Entry vs. Barriers to Exit
 - Productization of open source does create dependencies.
 - But these dependencies are manageable.
- Creativity grows in a robust community
 - Hundreds of contributions to OpenSolaris, GlassFish, Open Office, etc. by non-Sun community members.

Non-Sun Contributors to Open-ESB.org

- Adnet – Workflow Service Engine
- Bostech – TCP/IP Binding Component, Scheduler Service Engine
- Eviware – Soap UI Netbeans Plugin
- Gestalt-LLC – XMPP, SIP, UDDI & RSS Binding Components
- Imola – CICS & CORBA Binding Components, Semantic Web Service Engine
- Neociclo – Odette FTP, Odette-J Binding Components
- Vikram India – DCOM Binding Component
- Yasu – Business Rules Service Engine
- Art Gillespie – 3270 Binding Component
- Jon Strayer & Art Gillespie – Screen-scraping Binding Component
- Gary Zheng – LDAP Binding Component
- Oscar Montoya – SWIFT Binding Component
- Stelian Markov – eMail Binding Component

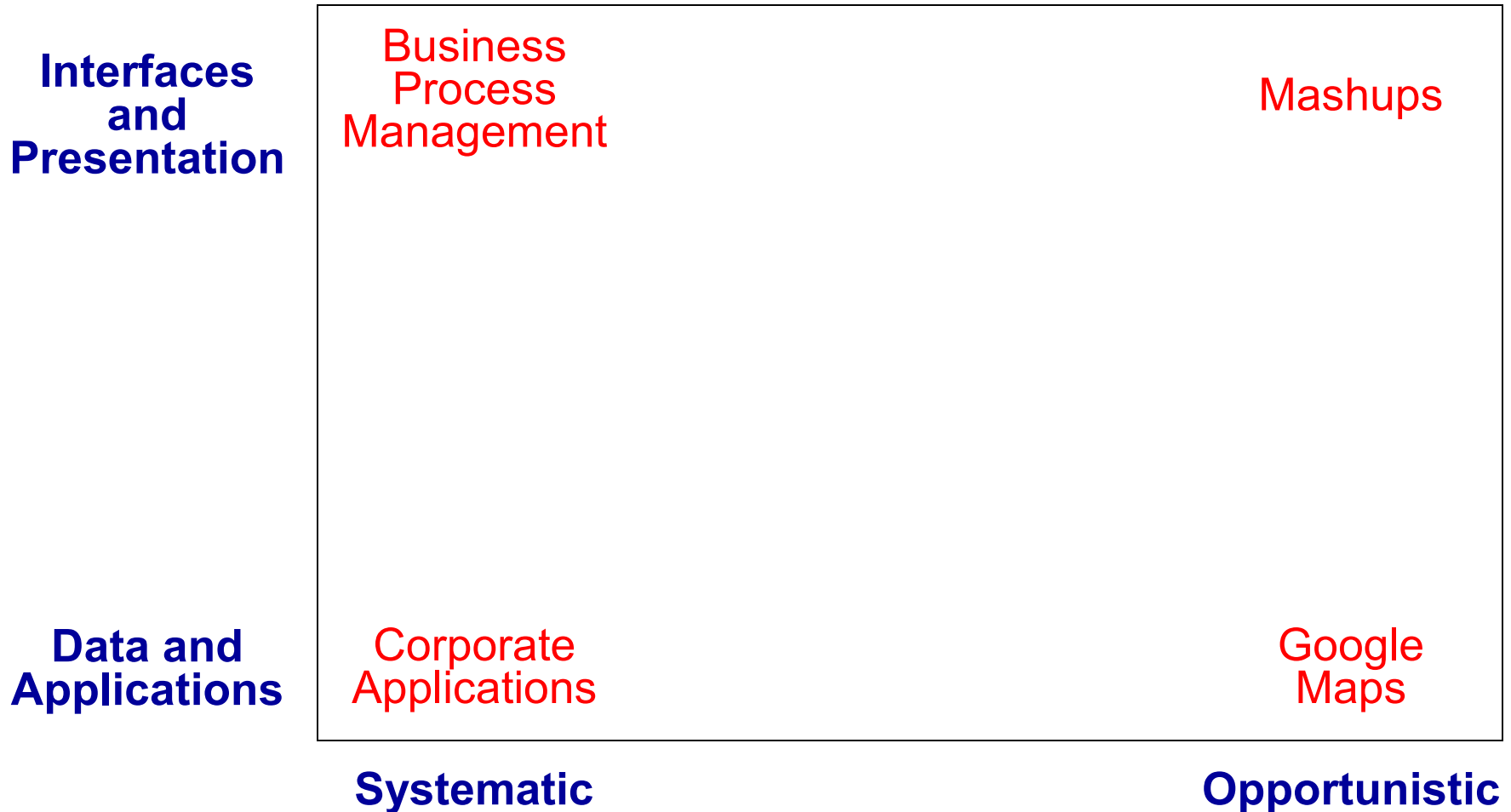
Sun's Commitment to Open Source

- Sun is the largest contributor of open source code in the world
- Everything that Sun has today is either open source or it's going through indemnification so that it can become open source.
- Sun contributes to open source projects in many domains: Open Office, MySQL, Open Solaris, GlassFish, NetBeans, Open ESB, Open Portal, Open SSO, Open DS, Derby, Open MQ, WoodStock, OpenInstaller . . .

Agenda

- Why should enterprises care about SOA?
- How are open standards and open source playing a role in the enterprise transition to SOA?
- **What are the biggest challenges around SOA?**
- What is the future of SOA?

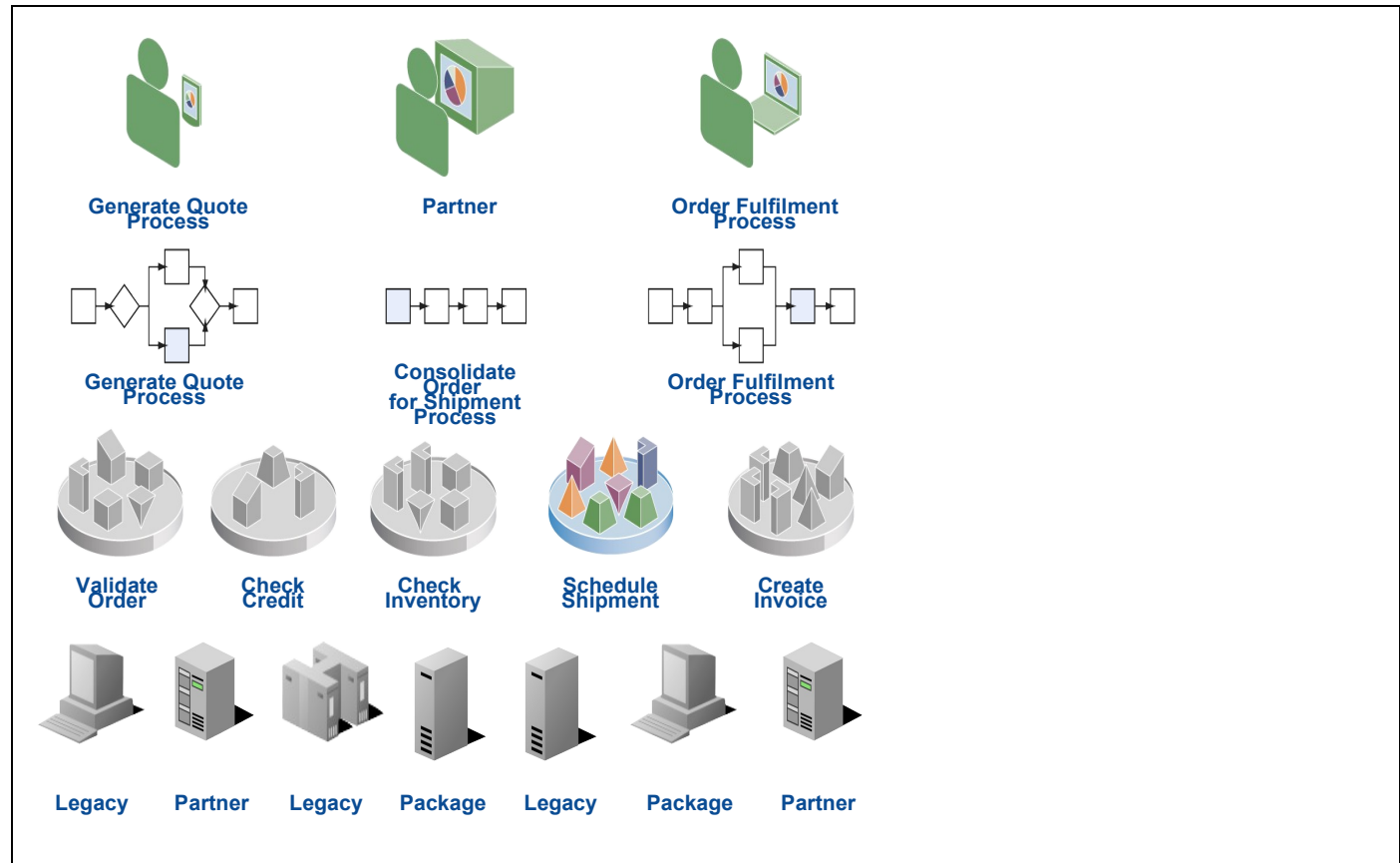
Methodology: Your development methodology will vary based on the demands of the application.



Run-the-business performance, availability, security and maintainability require a systematic approach.

Interfaces and Presentation

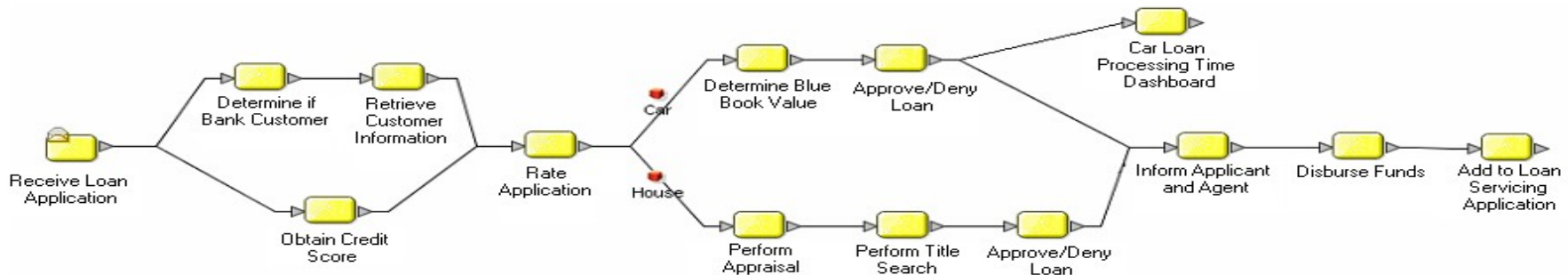
Data and Applications



Systematic

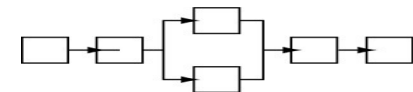
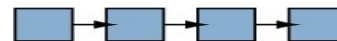
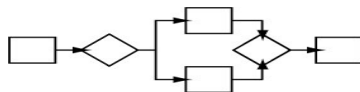
Opportunistic

Tooling: An ESB provides functionality needed to develop and manage services within an SOA.



Messaging and Request / Reply

Message Authentication, Access Management & Encryption



Orchestration

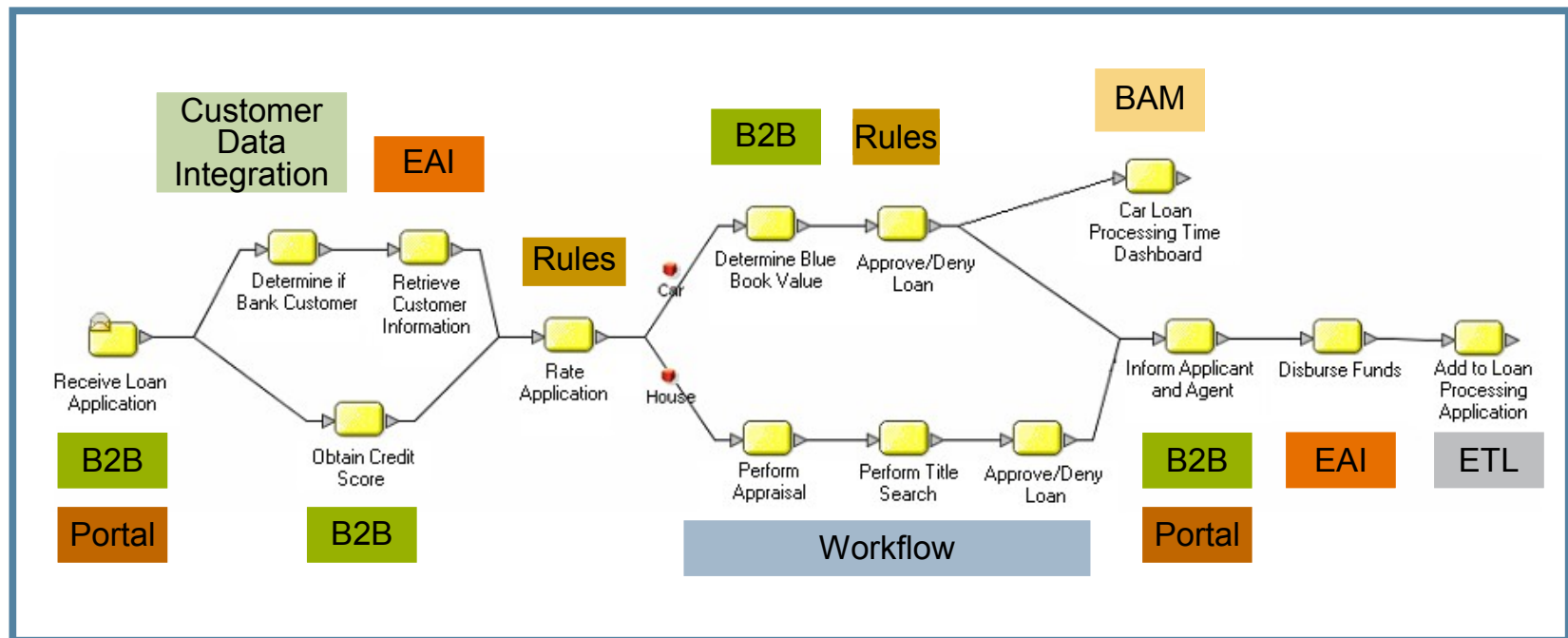


Transformation

Wrappers / Adapters



Most ESB Suites also provide technology to deliver Composite Applications on top of those services.



An integrated toolset reduces costs throughout the system development lifecycle.

- Advantages of a single repository
 - Better support for reuse.
 - Better support for impact analysis.
 - Better support for version control and configuration management.
- Advantages of a single runtime
 - A single management framework.
 - A single security framework
 - A single transaction management framework
 - A single debugging environment
- Advantages of a single development environment
 - A consistent look and feel
 - Reuse of development tools

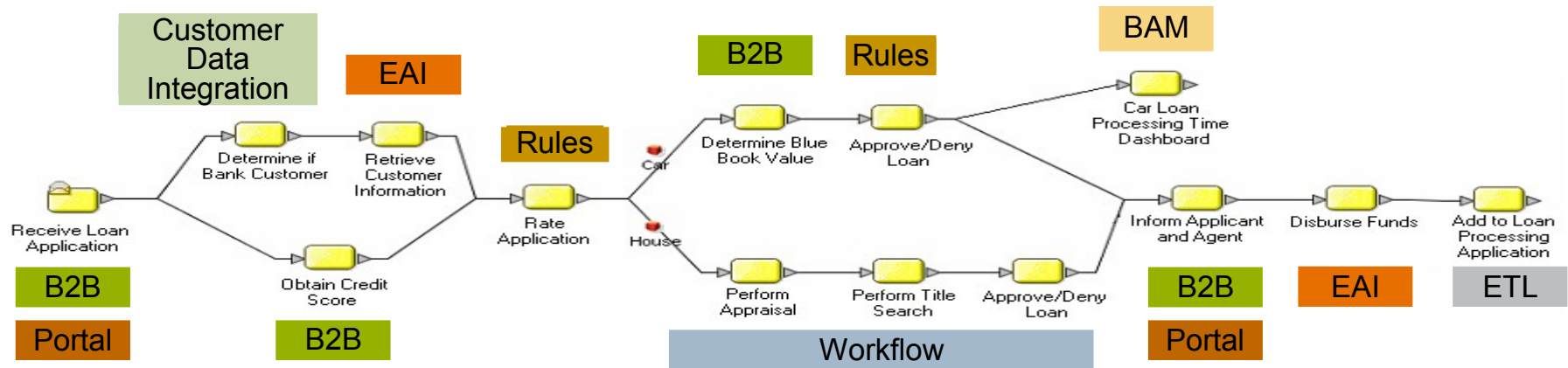
The Advantages of an Integrated Suite

The Butler Group conducted a customer study:

What are the savings that can be realized through use of an integrated development and deployment environment:

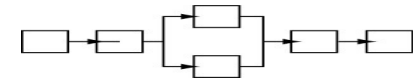
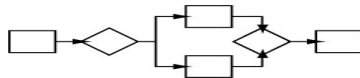


Security: Identity and Security have to be built into the fabric of the architecture.



Messaging and Request / Reply

Message Authentication, Access Management & Encryption



Orchestration



Transformation

Wrappers / Adapters

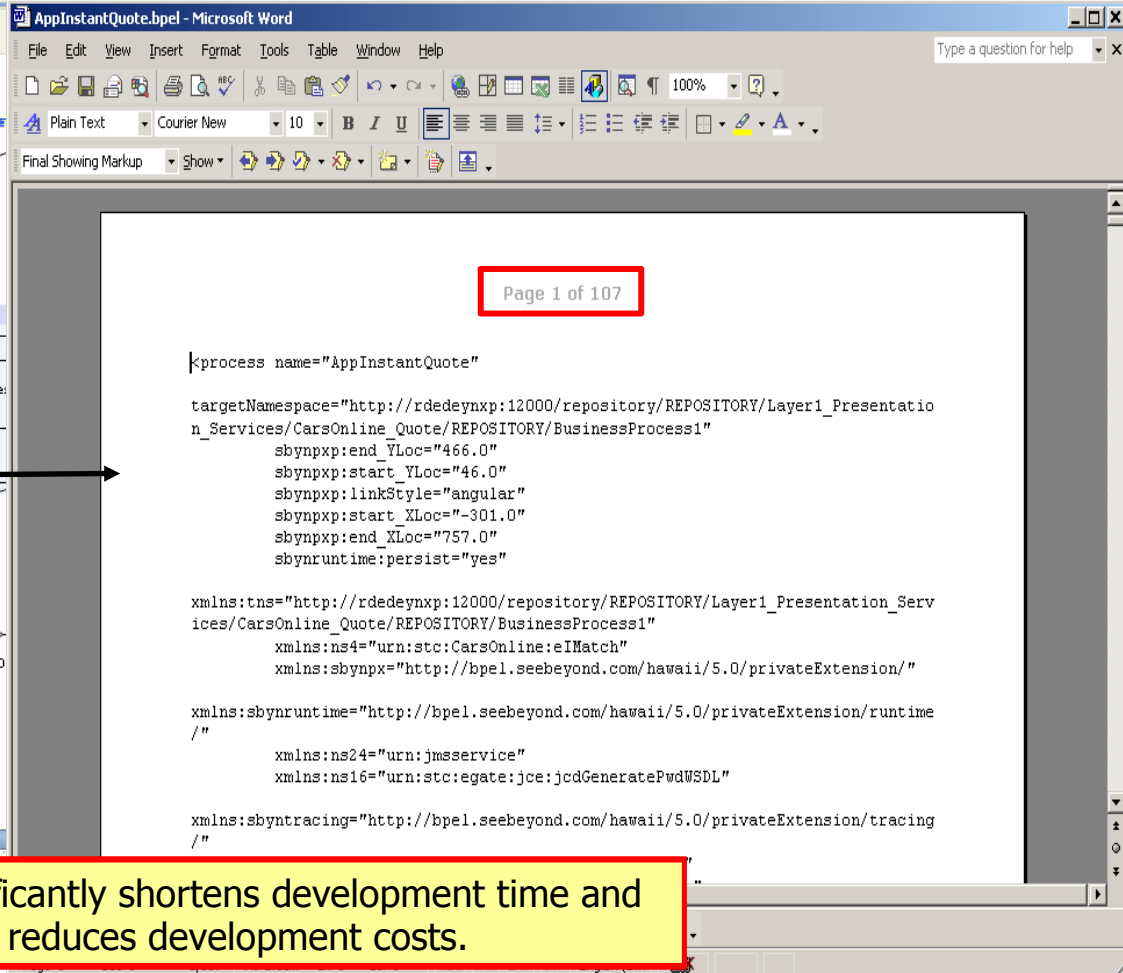
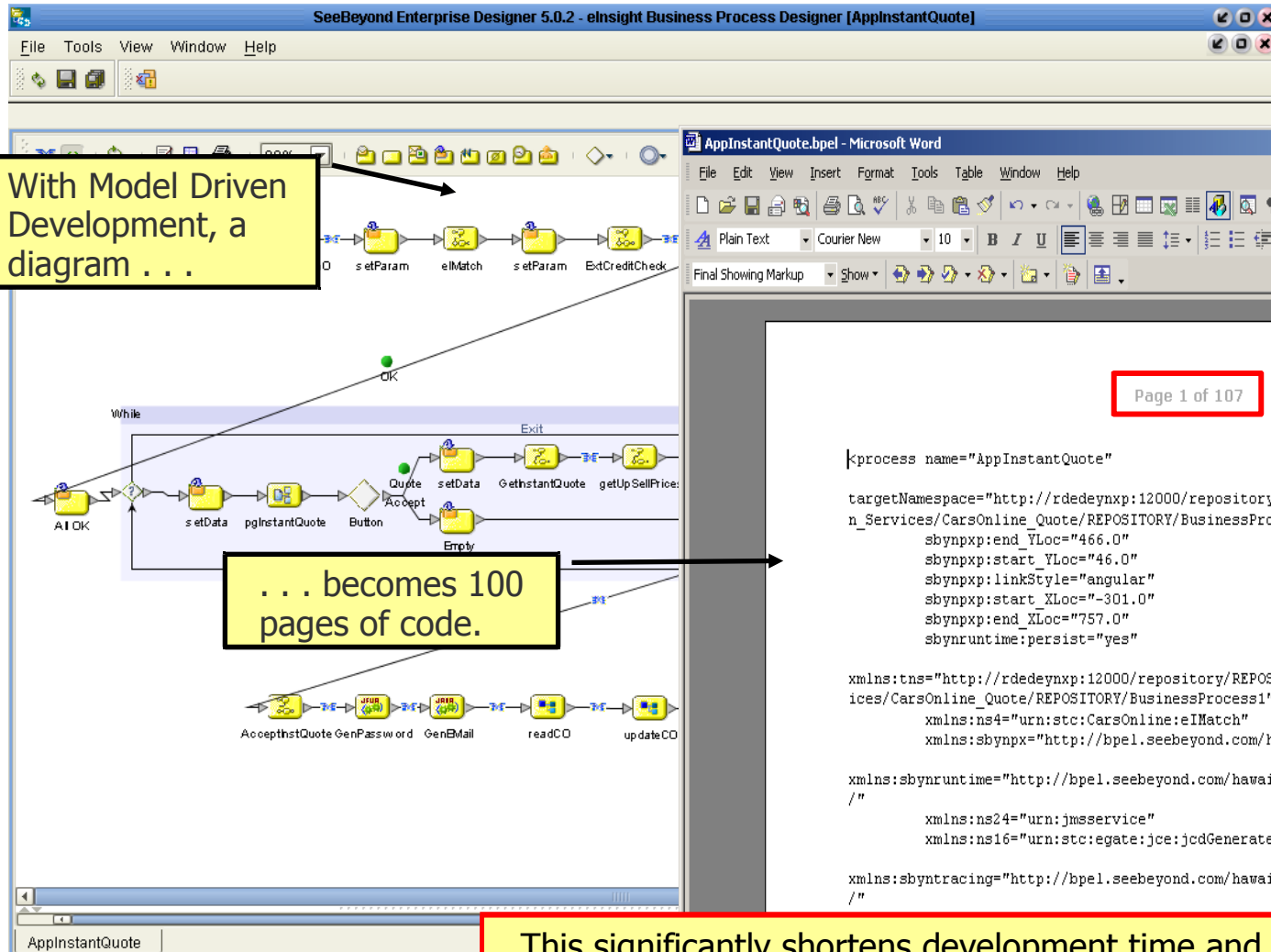


Development Effort: Model Driven Development

With Model Driven Development, a diagram . . .

. . . becomes 100 pages of code.

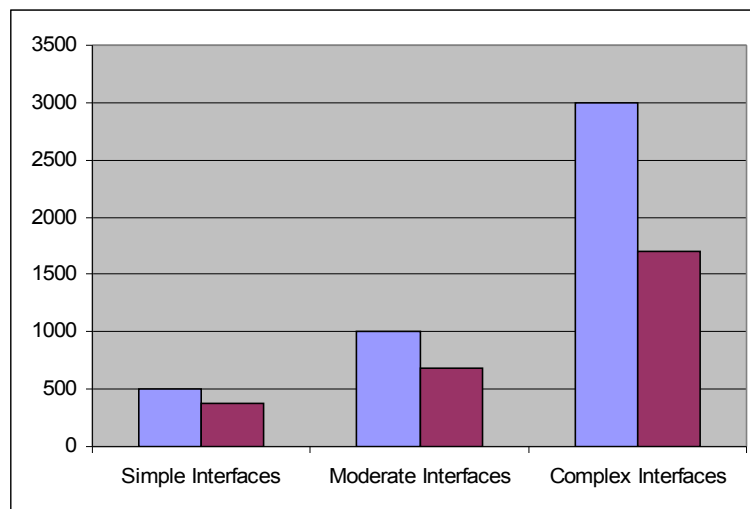
This significantly shortens development time and reduces development costs.



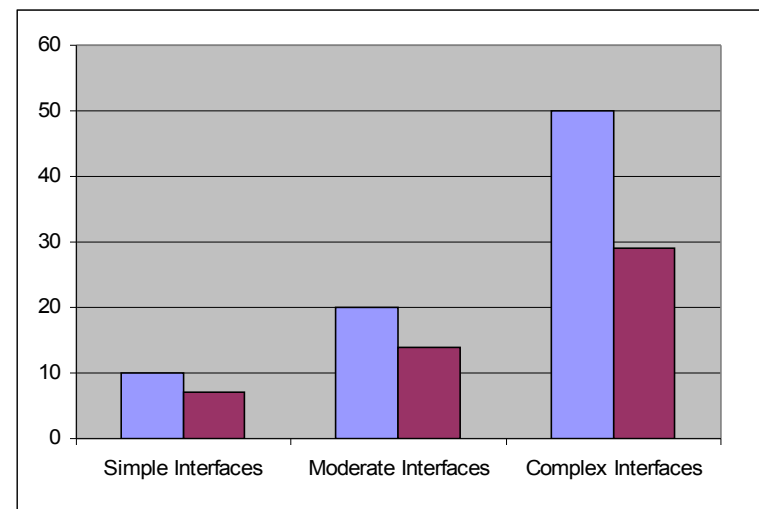
Model Driven Development reduces programming and maintenance costs for Composite Applications.

Model Driven Development also reduces “time to market” and manages development risk.

Number of Hours Per Interface



Maintenance Costs



Reduce the number of hours required to build a service by 25-43%.

Reduce the cost of maintaining that service by more than 25-43%.

Governance: Policy-Driven Runtime Management

- Reduce Complexity via Separation of Concerns
 - A Policy-centric approach to governance allows policy to be managed independently of the service runtime – reducing cost and disruption while increasing control and flexibility.
- Promote Responsible Reuse
 - The ability to govern service usage is essential for promoting re-use of business assets in a way that protects the interests of the service provider and the service consumer.
- Ensure Regulatory Compliance
 - Dynamic, conditional, multi-jurisdictional regulations for privacy and accountability pose additional challenges for architects of complex, heterogeneous multi-domain service networks.

Governance: “Design-Time” Governance

- An organization needs to secure services.
 - To begin, they decorate the service facade with WS-Security.
 - Later, they realize the service needs logging and alerting.
 - As they support reusable services over an extended lifecycle, they then see the need for service versioning.
 - Finally, they decide to initiate service-level billing and . . .
- . . . soon scalability is a problem – certainly runtime scalability, but, particularly, scalability of the development environment and process
 - So, they define a set of shared technical services, connect them to the ESB and allow the business services to leverage this shared set of technical services.
 - Once a few dozen business services begin talking to a few dozen governance services over the ESB, any required changes to these services create a state of chaos.
- As you can see, the need for a well designed Governance model to control your multiplying services is essential.

Performance Management and Optimization

- Optimization of runtime performance within an SOA has several elements:
 - Performance of each executed (legacy) program must be managed.
 - The application must continue to meet its SLAs for “legacy” users while scaling to meet the incremental transaction arrival rate required as a service endpoint.
 - Performance of each (newly developed, composite) service must be managed.
 - There are two factors which govern service performance: execution of code within the service and the network latency + application latency encountered when invoking external applications.
 - Performance of Composite Applications must be measured and managed.
 - There are two factors which govern service performance: execution of code within the service and the network latency + application latency encountered when invoking external applications.

Dynamically Tracing Application Components

- Because of its highly modular and distributed nature, a Composite Application is a challenge to profile
- Sometimes you just can't see the problem when you pull the application back to the test environment
- The ability to dynamically trace components across the entire application stack top-to-bottom / end-to-end is critical
- DTrace, luckily, runs on Apple OS/X, Solaris, Linux and BSD
- You can DTrace all of your tools and applications, including Postgres, MySQL, the GlassFish app server layer and Java code (including both handwritten Java code and Java-based components)
- DTrace produces zero overhead in production while it's turned off, then 3-4% overhead in production when you turn it on

Agenda

- Why should enterprises care about SOA?
- How are open standards and open source playing a role in the enterprise transition to SOA?
- What are the biggest challenges around SOA?
- **What is the future of SOA?**

The Future of SOA

- Real Time SOA – based on a Real Time OS / Real Time Java
- Mobile SOA – requires flexible client-side support; federated identity
- Trusted SOA – requires an OS with Trusted Extensions
- Plug-and-Play SOA
 - “Plug-and-Play Integration” – will be incredibly rare
 - “Plug-and-Play Interoperability” – we can get there, but there’s no guarantee
 - Intensive verticalization is key to any Plug-and-Play SOA initiative
- Complex Event Processing / Business Activity Monitoring / Intelligent Event Processing – similar in some ways to SOA; leverages some SOA infrastructure
- Model Driven SOA – generating code that is visible and works
- Scalable SOA – scalability at runtime, but also scalable development

Thank You!

Ross Altman

Sun Microsystems

Ross.Altman@Sun.com

Positioning the Vendors in the ESB Market

- **Place Holder Functionality**
- **Point Products**
- **Open Source and Open Standards**

